HDOC Day-54 Activity

Maximum Sum of a Contiguous Subarray of Size k Using Pointers

Student Name	Dhruv Pandey
SAP ID	590037660
Batch	15
Course / Activity	HDOC Day-54

Aim

To write a C program that finds the maximum sum among all contiguous subarrays of size **k**. The input array is passed to **max_subarray_sum()** using its address and accessed using pointers.

Problem Requirements

- Read the size and elements of a variable-length integer array **arr[]**.
- Read an integer **k**.
- Pass the array to **max_subarray_sum()** using its address.
- Identify every contiguous subarray containing exactly **k** elements.
- Calculate the sum of each such subarray and find the maximum.
- Display **-1** when k is invalid.

Algorithm

1. Read the array size **n**, its elements, and **k**.
2. Pass **arr**, **n**, and **k** to **max_subarray_sum()**.
3. If **k <= 0** or **k >= n**, display -1 and return.
4. Use an outer loop from index 0 through **n-k**. Each position represents the beginning of one contiguous subarray.
5. For each starting position, use an inner loop of **k** elements and calculate its sum using pointer notation.
6. Store the first sum as the initial maximum and update it whenever a larger sum is found.
7. Display the maximum subarray sum.

C Program

```c
#include <stdio.h>

void max_subarray_sum(int *arr, int n, int k)
{
    if (k <= 0 || k >= n)
    {
        printf("-1");
        return;
    }

    int max_sum = 0;

    for (int i = 0; i <= n - k; i++)
    {
        int sum = 0;

        for (int j = 0; j < k; j++)
        {
            sum = sum + *(arr + i + j);
        }

        if (i == 0 || sum > max_sum)
        {
            max_sum = sum;
        }
    }

    printf("Maximum subarray sum: %d", max_sum);
}

int main()
{
    int n, k;

    printf("Enter the size of array: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter the elements: ");
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    printf("Enter k: ");
    scanf("%d", &k);

    max_subarray_sum(arr, n, k);

    return 0;
}
```

Pointer Usage

max_subarray_sum(arr, n, k) passes the address of the first array element. The parameter **int *arr** receives this address. Inside the nested loops, ***(arr + i + j)** accesses the required element of each contiguous subarray.

Example 1 Dry Run

Subarray	Calculation	Sum

2 1 5	2 + 1 + 5	8
1 5 1	1 + 5 + 1	7
5 1 3	5 + 1 + 3	9
1 3 2	1 + 3 + 2	6

The largest value is **9**, produced by the contiguous subarray **5 1 3**.

Test Cases / Expected Results

Case	Array	k	Expected Output
Example 1	2 1 5 1 3 2	3	Maximum subarray sum: 9
Example 2	2 3 4 1 5	2	Maximum subarray sum: 7
Example 3	1 2 3	4	-1

Example 2 Explanation

For array **2 3 4 1 5** and k = 2, the subarray sums are: 2+3=5, 3+4=7, 4+1=5, and 1+5=6. Therefore, the maximum subarray sum is **7**.

Example 3 - Invalid k

For an array of size 3, k = 4 is invalid because k is greater than the array size. Therefore, the program displays **-1**.

Complexity

Time Complexity: O((n-k+1) x k), which is O(nk) in general, because nested loops calculate the sum of each size-k subarray.

Space Complexity: O(1) extra space because only variables for the current sum and maximum sum are required.

Result

The program successfully identifies all contiguous subarrays of size k, calculates their sums using pointer access, finds the maximum sum, and displays -1 when k is invalid.

Execution Example

```
Enter the size of array: 6
Enter the elements: 2 1 5 1 3 2
Enter k: 3
Maximum subarray sum: 9
```